

---

## Chapitre n° 7

---

### ALGORITHMES GLOUTONS

---

Un problème d'optimisation a deux caractéristiques : une fonction que l'on doit maximiser ou minimiser et une série de contraintes auxquelles il faut satisfaire. On peut essayer de résoudre ce type de problème en écrivant un algorithme qui énumère les possibilités de manière exhaustive afin de trouver la meilleur. C'est un algorithme très simple mais souvent inutilisable en machine à cause de son coût. L'objectif d'un algorithme glouton (en anglais greedy algorithm) est d'obtenir une solution rapidement. Mais celle-ci n'est pas toujours la solution optimale.

Un choix peut être globalement optimal, c'est le meilleur de tous, ou localement optimal, c'est le meilleur parmi un ensemble restreint de choix. A chaque étape exécutée par un algorithme, se présente un ensemble de choix et un algorithme glouton fait le meilleur choix parmi les propositions. Un choix glouton est donc un choix localement optimal. La question est de savoir si en faisant une série de choix localement optimaux, on fini par aboutir à une solution optimale. C'est parfois le cas mais pas toujours.

#### I PROBLÈME DU RENDU DE MONNAIE

Pour simplifier nous supposons que nous n'avons que des pièces. Un système de pièces est alors un  $n$ -uplet  $S = (p_0, p_1, \dots, p_{n-1})$ , où  $p_i$  représente la valeur de la pièce d'indice  $i$ . Ces valeurs constituent une suite de nombres entiers strictement croissante avec  $p_0 = 1$ . Si  $p_0 > 1$ , alors certaines sommes ne peuvent pas être rendues.

Le problème du rendu de monnaie consiste à trouver une liste d'entiers positifs  $[x_0, x_1, \dots, x_{n-1}]$  qui vérifie  $x_0 p_0 + x_1 p_1 + \dots + x_{n-1} p_{n-1} = r$  où  $r$  est la somme à rendre en minimisant la somme  $x_0 + x_1 + \dots + x_{n-1}$ , c'est-à-dire le nombre de pièces utilisées. La condition  $x_0 p_0 + x_1 p_1 + \dots + x_{n-1} p_{n-1} = r$  est appelée contrainte. Dans le système de la zone euro, par exemple, nous avons en centimes les pièces suivantes :  $S = (1, 2, 5, 10, 20, 50, 100, 200, 500)$  où 100, 200 et 500 représentent respectivement les pièces de 1, 2 et 5 euros.

Il y a de nombreuses manières de rendre huit centimes. Les pièces qui peuvent être utilisées sont les pièces de 1, 2 et 5 centimes. Nous n'écrivons que les triplets possibles :  $[8, 0, 0]$ ,  $[6, 1, 0]$ ,  $[4, 2, 0]$ ,  $[3, 0, 1]$ ,  $[2, 3, 0]$ ,  $[1, 1, 1]$  et  $[0, 4, 0]$ . Un programme permet de tester de manière exhaustive tous les triplets :

```
1 p=(1,2,5)
2 for i in range(9):
3     for j in range(5):
4         for k in range(2):
5             s=i * p[0]+j * p[1]+k * p[2]
6             if s==8:
7                 print ([i, j, k])
```

On constate que le triplet qui minimise le nombre de pièces est le triplet  $[1, 1, 1]$  avec 3 pièces. Mais c'est "long". La méthode gloutonne consiste à pour un rendu  $r$  :

- chercher par valeur décroissante en partant de la pièce qui a la plus forte valeur, la première pièce, qui a une valeur inférieure ou égale à la somme à rendre  $r$ .
- recommencer en partant de la pièce prise en cherchant celle qui a une valeur inférieure ou égale à la nouvelle somme à rendre  $r - v$ .
- et ainsi de suite jusqu'à arriver à une somme à rendre nulle.

Les entrées sont  $p$  pour le système de pièces et  $r$  pour la somme à rendre.

```

1 def monnaie(p,r):
2     n = len(p)
3     i = n - 1
4     solution = n*[0]
5     while r > 0 :
6         while p[i]> r :
7             i = i- 1
8         solution[i] = solution[i] + 1
9         r = r - p[i]
10    return solution

```

**EXERCICE 1.** Tester à la main le rendu de monnaie, avec  $r = 8$  et  $p = (1, 2, 5, 10, 20, 50, 100, 200, 500)$ .

Dans le cas général, avec un système différent de celui montré en exemple, c'est un problème difficile à résoudre. Mais dans presque tous les systèmes de monnaie, l'algorithme glouton est optimal. Pour rendre la monnaie, on rend la pièce (ou le billet) de valeur maximale, et on continue tant qu'il reste quelque chose à rendre.

Le système monétaire utilisé en Angleterre jusque dans les années 1960 comportait une multitude de pièces : des pièces de 1 penny, 3 pence, 4 pence, 6 pence, 12 pence soit 1 shilling, etc. (Le mot penny est le singulier de pence.)

**EXERCICE 2.** Tester de rendre 8 pence avec ce système. La solution gloutonne est-elle optimale? (Et non je ne n'acharne pas)

## II PROBLÈME DU SAC À DOS

Nous considérons la variante « entière » du problème du sac à dos.

Nous sommes devant un ensemble de  $n$  objets. Chaque objet noté  $o_i$  a une valeur notée  $v_i$  et un poids noté  $p_i$ . Il s'agit d'emporter dans son sac à dos l'ensemble d'objets qui a la plus grande valeur sachant que le sac supporte un poids maximum  $P$ . Comment résoudre ce problème, quels objets doit-on prendre?

Pour appliquer une stratégie gloutonne, nous devons définir ce que nous entendons par le meilleur choix à chaque étape.

Il y a trois manières ici de définir un meilleur choix. Parmi les objets qui n'ont pas encore été pris, soit on choisit un objet qui a la valeur maximale, soit un objet qui a le poids minimal, soit un objet qui a le rapport valeur/poids maximal.

Nous ne considérons que les objets ayant un poids  $p_i \leq P$ . L'algorithme glouton consiste, à chaque étape, à choisir parmi ces objets celui qui représente le choix optimal. Nous le notons  $O1$ , sa valeur  $V1$  et son poids  $P1$ . Ensuite, nous recommençons parmi les objets de poids  $p_i \leq P - P1$ . Et ainsi de suite.

Cette variante est dite entière parce que chaque objet est pris ou pas. Il existe une version fractionnaire. Le principe de base est le même, mais cette fois il est possible de prendre des fractions d'objets. Un algorithme glouton donne une solution optimale à cette variante fractionnaire.

Prenons un exemple : le sac à dos peut contenir 15Kgs. Les poids des objets sont en Kg, les valeurs en euro.

| Objet   | Valeur | Poids | Valeur/Poids |
|---------|--------|-------|--------------|
| Objet 1 | 126    | 14    | 9            |
| Objet 2 | 32     | 2     | 16           |
| Objet 3 | 20     | 5     | 4            |
| Objet 4 | 5      | 1     | 5            |
| Objet 5 | 18     | 6     | 3            |
| Objet 6 | 80     | 8     | 10           |

Un objet est représenté par une liste comme [l'objet 1', 126, 14].

Nous définissons trois fonctions qui prennent en paramètre un objet et renvoient respectivement la valeur, l'inverse du poids et le rapport valeur/poids de l'objet. Remarque : si le poids est minimal, son inverse est maximal.

```
1 def valeur(obj):
2     return obj[1]
3
4 def invpoids(obj):
5     return 1/obj[2]
6
7 def rapport(obj):
8     return obj[1]/obj[2]
```

Nous définissons ensuite une fonction glouton qui prend en paramètres une liste d'objets, un poids maximal (celui que peut supporter le sac à dos) et le type de choix utilisé (par valeur, par poids, ou par valeur/poids).

La première chose à faire est de trier la liste suivant le type de choix utilisé par ordre décroissant. C'est le principe du TP sur les tris, trier une liste selon un certain critère. Nous utilisons la fonction `sorted` de python via l'appel `sorted(liste, key=choix, reverse = True)`.

```
1 def glouton(liste, poids_max, choix):
2     copie = sorted(liste, key=choix, reverse = True)
3     reponse = []
4     valeur = 0
5     poids = 0
6     i = 0
7     while i < len(liste) and poids <= poids_max :
8         nom, val, pds = copie[i]
9         if poids + pds <= poids_max :
10             reponse.append(nom)
11             poids = poids + pds
12             valeur = valeur + val
13         i = i+1
14     return reponse, valeur
```

Il reste à définir la liste d'objets puis à exécuter la fonction glouton en précisant le type de choix utilisé.

```
1 objets = [['objet 1', 126,14], ['objet 2', 32,2], ['objet 3', 20,5], ['objet 4', 5,1],
2           ['objet 5', 18,6], ['objet 6', 80,8]]
3
4 print(glouton(objets,15,valeur))
5 print(glouton(objets,15,invpoids))
6 print(glouton(objets,15,rapport))
```

**EXERCICE 3.** Tester le tout! Quel semble le meilleur choix?

**EXERCICE 4.** Est-ce une solution optimale?

Dans une version fractionnaire du problème où l'on peut choisir une fraction du poids maximal autorisé pour chaque objet (par exemple s'il s'agit de poudre ou de liquide), le meilleur choix est dicté par le critère valeur/poids. Une stratégie gloutonne ferait donc prendre en premier les 2 kilos d'objet 2, puis les 8 kilos d'objet 6 et 5 kilos d'objet 1. Dans ce cas, nous obtenons finalement une valeur totale de  $32 + 80 + 45 = 157$ .

### III LIMITES ET SOLUTIONS

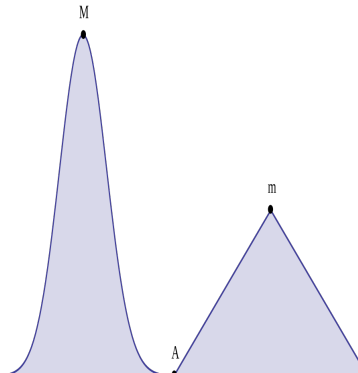
De manière générale un algorithme glouton n'est pas là pour donner une solution optimale, mais une solution rapide (ou du moins une solution tout court quand même en trouver une est complexe), qu'on espère assez proche d'une solution optimale.

Par exemple si on cherche le maximum global dans le cadre ci contre, en partant de  $A$  et suivant la plus forte pente, on arrivera en  $m$  maximum local mais pas global.

Il est extrêmement compliqué de parler de problèmes d'optimisation et des méthodes diverses (c'est le but d'un master...)

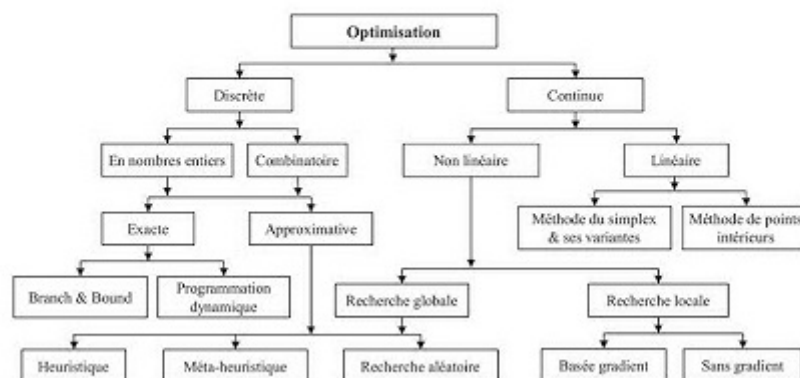
Mais l'idée dans ce cas est : comment se sortir de ce maximum local ?

- en perturbant notre solution ?
- en acceptant de revenir en arrière ?
- en se plaçant au hasard ?
- autre ?



C'est un sujet extrêmement vaste, qu'on effleurera au travers d'exemples.

### Classification des méthodes d'optimisation



source dans le tp.